

DESARROLLO DE APLICACIONES MULTIPLATAFORMA (DAM)

*Programación Multimedia y Dispositivos Móviles · 2025/2026*

---

# PRÁCTICA 2

---

## Creación del Proyecto Android Studio – GearLog

---

*SplashScreen · Activity · Intent implícito · Análisis del Proyecto*

Alumno/a: Valentín Urdillo

Ciclo: 2º DAM – Desarrollo de Aplicaciones Multiplataforma

Curso: 2025 / 2026

Fecha: \_\_\_\_\_

## Índice

---

- 1. Descripción del proyecto – GearLog**
- 2. Estructura del proyecto en Android Studio**
- 3. Análisis del proyecto – Preguntas y respuestas**
  - P.1 Ruta absoluta al directorio principal
  - P.2 Ruta relativa al directorio de actividades
  - P.3 Ruta relativa al directorio de recursos
  - P.4 Elementos de la SplashScreen
  - P.5 Clases creadas y herencia
  - P.6 Composables en Jetpack Compose
  - P.7 Función onCreate
  - P.8 Uso del objeto Intent
  - P.9 Fichero strings.xml
  - P.10 Directorio mipmap e ic\_launcher.xml
  - P.11 AndroidManifest.xml
  - P.12 Cláusulas del manifiesto
- 4. Webgrafía**

## 1. Descripción del proyecto – GearLog

GearLog es una red social orientada a aficionados y profesionales de la mecánica y la automoción. La idea surge de la necesidad de tener un espacio donde los usuarios puedan documentar y compartir reparaciones, diagnósticos de averías, proyectos de restauración de vehículos clásicos y trucos de taller.

| Campo            | Valor                                                                    |
|------------------|--------------------------------------------------------------------------|
| Nombre de la app | GearLog                                                                  |
| Temática         | Red social de mecánica y automoción                                      |
| Versión          | 1.0.0                                                                    |
| Package          | com.valentinurdillo.gearlog                                              |
| Min SDK          | API 26 (Android 8.0 Oreo)                                                |
| Target SDK       | API 35 (Android 15)                                                      |
| Lenguaje         | Kotlin + Jetpack Compose                                                 |
| API pública      | NHTSA Vehicle API ( <a href="https://api.nhtsa.gov/">api.nhtsa.gov</a> ) |
| Desarrollador    | Valentín Urdillo                                                         |

La aplicación en su versión actual incluye una SplashScreen de bienvenida y una pantalla 'Sobre la Aplicación' con los datos del proyecto y un acceso directo para contactar por correo electrónico. En futuras prácticas se añadirán las funcionalidades de red social: feed, publicaciones, comentarios, perfil de usuario y consulta de datos de vehículos via API.

A  
P  
l  
i  
c  
a  
c  
i  
o  
n  
:  
N  
H  
T  
S  
A  
V  
e  
h  
i  
c  
l  
e  
s

A  
P  
I  
—  
h  
t  
t  
p  
s  
:  
/  
/  
a  
p  
i  
.  
n  
h  
t  
s  
a  
.  
g  
o  
v  
—  
A  
P  
I  
p  
ú  
b  
l  
i  
c  
a  
y  
g  
r  
a  
t  
u  
i  
t  
a  
d  
e  
l  
D  
e  
p  
a  
r  
t  
a  
m  
e  
n  
t

o  
d  
e  
T  
r  
a  
n  
s  
p  
o  
r  
t  
e  
d  
e  
E  
E  
·  
U  
U  
·  
q  
u  
e  
o  
f  
r  
e  
c  
e  
d  
a  
t  
o  
s  
d  
e  
r  
e  
c  
a  
l  
i  
s  
,  
r  
a  
t  
i  
n  
g  
s  
d  
e  
s  
e  
g  
u  
r

i  
d  
a  
d  
y  
e  
s  
p  
e  
c  
i  
f  
i  
c  
a  
c  
i  
o  
n  
e  
s  
d  
e  
v  
e  
h  
í  
c  
u  
l  
o  
s  
.  
N  
o  
r  
e  
q  
u  
i  
e  
r  
e  
c  
l  
a  
v  
e  
d  
e  
A  
P  
I  
.

## 2. Estructura del proyecto en Android Studio

---

Al crear el proyecto con la plantilla Empty Activity (Compose) en Android Studio, se genera automáticamente la siguiente estructura de directorios y ficheros:

```
G  
e  
a  
r  
L  
o  
g  
/  
-  
-  
a  
p  
p  
/  
-  
-  
s  
r  
c  
/  
-  
-  
m  
a  
i  
n  
/  
-  
-  
j  
a  
v  
a  
/  
c  
o  
m  
/  
v  
a  
l  
e  
n  
t  
i  
n
```

```
n  
u  
r  
d  
i  
l  
l  
o  
/  
g  
e  
a  
r  
l  
o  
g  
/  
-  
-  
-  
M  
a  
i  
n  
A  
c  
t  
i  
v  
i  
t  
y  
.  
k  
t  
-  
-  
-  
S  
p  
l  
a  
s  
h  
A  
c  
t  
i  
v  
i  
t  
y  
.  
k  
t  
-
```



u i / t h e m e / T h e m e . k t r e s / d r a w a b l e / i c l a

```
un-  
ch-  
er-  
|  
b-  
a-  
c-  
k-  
g-  
r-  
o-  
u-  
n-  
d-  
.  
x-  
m-  
l-  
|  
|  
i-  
c-  
|  
l-  
a-  
u-  
n-  
c-  
h-  
e-  
r-  
|  
f-  
o-  
r-  
e-  
g-  
r-  
o-  
u-  
n-  
d-  
.  
x-  
m-  
l-  
|  
|  
|  
m-  
i-  
p
```

```
m
a
p
-
a
n
y
d
p
i
-
v
2
6
/
|
|
|
i
c
|
l
a
u
n
c
h
e
r
.
x
m
l
|
|
|
v
a
l
u
e
s
/
|
|
|
s
t
r
i
n
g
```

```

s . x m l | - t h e m e s . x m l | - x m l / | - b a c k u p - r u l e s . x m l | - d a t

```

```
a-  
e-  
x-  
t-  
r-  
a-  
c-  
t-  
i-  
o-  
n-  
-  
r-  
u-  
l-  
e-  
s-  
.  
x-  
m-  
l-  
-  
-  
A-  
n-  
d-  
r-  
o-  
i-  
d-  
M-  
a-  
n-  
i-  
f-  
e-  
s-  
t-  
.  
x-  
m-  
l-  
-  
-  
b-  
u-  
i-  
l-  
d-  
.  
g-  
r-  
a-  
d-  
l-  
e-  
.
```

```
k  
t  
s  
-  
-  
g  
r  
a  
d  
l  
e  
/  
-  
-  
l  
i  
b  
s  
.  
v  
e  
r  
s  
i  
o  
n  
s  
.  
t  
o  
m  
l  
-  
-  
b  
u  
i  
l  
d  
.  
g  
r  
a  
d  
l  
e  
.  
k  
t  
s  
-  
-  
s  
e  
t  
t  
i  
n
```

```
gs
  .
  gradle
  .
  kts
  L
  -
  -
  gradle
  .
  properties
  .
```

### 3. Análisis del Proyecto – Preguntas y Respuestas

---

**P.1**

**Ruta absoluta al directorio principal de la aplicación**

**0,5 pts**

#### ¿Qué es una ruta absoluta?

Una ruta absoluta es aquella que indica la ubicación completa de un fichero o directorio partiendo desde el directorio raíz del sistema de archivos. No depende de cuál sea el directorio de trabajo actual, ya que siempre parte del mismo punto de origen (la raíz). En sistemas Windows el origen es la unidad de disco (por ejemplo C:\), mientras que en Linux/macOS es la barra inclinada (/).

#### Ruta absoluta del proyecto GearLog

En un sistema Windows con Android Studio instalado con la configuración por defecto, la ruta absoluta al directorio raíz del proyecto sería:

```
C:\Users\Valentin\AndroidStudioProjects
```



```
\
G
e
a
r
L
o
g
```

Dentro de esa ruta, el directorio principal del módulo app, donde se encuentra todo el código fuente y los recursos de la aplicación, es:

```
C
:
\
U
s
e
r
s
\
V
a
l
e
n
t
i
n
\
A
n
d
r
o
i
d
S
t
u
d
i
o
P
r
o
j
e
c
t
s
\
G
e
a
r
L
o
g
\
a
```

pp/  
src/  
main

En  
Android  
Studio,  
puedes  
consultar  
la  
ruta  
absoluta  
de

c  
u  
a  
l  
q  
u  
i  
e  
r  
f  
i  
c  
h  
e  
r  
o  
h  
a  
c  
i  
e  
n  
d  
o  
c  
l  
i  
c  
d  
e  
r  
e  
c  
h  
o  
s  
o  
b  
r  
e  
é  
l  
e  
n  
l  
a  
v  
i  
s  
t  
a  
d  
e  
p  
r  
o  
y  
e  
c

t  
o  
—  
C  
o  
p  
y  
P  
a  
t  
h  
/  
C  
o  
p  
y  
R  
e  
f  
e  
r  
e  
n  
c  
e  
—  
A  
b  
s  
o  
l  
u  
t  
e  
P  
a  
t  
h  
.

**P.2****Ruta relativa al directorio de actividades y su importancia****0,5 pts**

Una ruta relativa indica la ubicación de un fichero o directorio tomando como punto de partida el directorio actual (normalmente la raíz del proyecto). Es más corta y portátil que la ruta absoluta porque no depende de dónde esté instalado el proyecto en el equipo.

**Ruta relativa al directorio de actividades**

a  
p  
p  
/  
s  
r

```
c  
/  
m  
a  
i  
n  
/  
j  
a  
v  
a  
/  
c  
o  
m  
/  
v  
a  
l  
e  
n  
t  
i  
n  
u  
r  
d  
i  
l  
l  
o  
/  
g  
e  
a  
r  
l  
o  
g  
/  

```

Este directorio contiene los ficheros Kotlin con las clases de la aplicación. En el proyecto GearLog encontramos:

- MainActivity.kt — Activity principal que muestra la pantalla 'Sobre la Aplicación'.
- SplashActivity.kt — Activity de presentación que se ejecuta al arrancar la app.
- ui/theme/Theme.kt — Definición del tema visual de la aplicación (colores, tipografía).

Este directorio es el núcleo de la lógica de la aplicación. Cada fichero .kt define una clase que hereda de ComponentActivity (o sus superclases) y controla el comportamiento de una pantalla. Sin este directorio la aplicación no tendría ningún comportamiento ni pantalla que mostrar.

|            |                                                                  |         |
|------------|------------------------------------------------------------------|---------|
| <b>P.3</b> | <b>Ruta relativa al directorio de recursos y su organización</b> | 0,5 pts |
|------------|------------------------------------------------------------------|---------|

```
a  
p  

```

```
p
/
s
r
c
/
m
a
i
n
/
r
e
s
/
```

El directorio `res/` es donde Android almacena todos los recursos estáticos de la aplicación. La gran ventaja de separar recursos del código es que Android puede seleccionar automáticamente el recurso más adecuado según la configuración del dispositivo (idioma, densidad de pantalla, tamaño, orientación, etc.).

| Subdirectorio                 | Contenido                                                                     |
|-------------------------------|-------------------------------------------------------------------------------|
| <b><code>drawable/</code></b> | Imágenes vectoriales XML, fondos, formas y selectores                         |
| <b><code>mipmap-*/</code></b> | Iconos del launcher en distintas densidades (mdpi, hdpi, xhdpi...)            |
| <b><code>values/</code></b>   | Ficheros XML con cadenas (strings.xml), colores, dimensiones, temas           |
| <b><code>xml/</code></b>      | Ficheros de configuración: backup_rules.xml, data_extraction_rules.xml        |
| <b><code>layout/</code></b>   | Ficheros XML de interfaz (en proyectos View; en Compose se definen en Kotlin) |
| <b><code>anim/</code></b>     | Definición de animaciones                                                     |
| <b><code>raw/</code></b>      | Ficheros de audio, vídeo u otros recursos sin procesar                        |

#### P.4 Elementos de la SplashScreen y su configuración

0,5 pts

La SplashScreen de GearLog se implementa mediante una Activity dedicada llamada `SplashActivity`. Se usa el enfoque nativo de Jetpack Compose con animaciones propias, dado que la API SplashScreen de Android 12+ gestiona el splash antes de que la Activity se cargue visualmente, pero para versiones anteriores (minSdk 26) se mantiene control manual.

#### Elementos incluidos en la SplashScreen

- Logo de la aplicación: icono de engranaje (⚙️) con tamaño 96sp, representando la temática de mecánica.
- Nombre de la app: texto 'GearLog' en 48sp, negrita, color blanco.
- Tagline: 'Tu taller en el bolsillo' en rojo corporativo (#E94560).

- Fondo: degradado vertical oscuro de #1A1A2E a #0F3460 (azul noche).
- Animación de aparición: fade-in + scale desde 0,7x hasta 1x (EaseOutBack), duración 800ms.

### Configuración básica

La duración total de la SplashScreen es de 1,8 segundos. Se implementa con LaunchedEffect y una corrutina que espera ese tiempo antes de lanzar la MainActivity mediante un Intent explícito:

```
L  
a  
u  
n  
c  
h  
e  
d  
E  
f  
f  
e  
c  
t  
(  
U  
n  
i  
t  
)  
{  
  
v  
i  
s  
i  
b  
l  
e  
=  
t  
r  
u  
e  
  
d  
e  
l  
a  
y  
(  
1  
8  
0  
0  
L  
)  
/  
/  
1  
,  
8  
s
```

```
e  
g  
u  
n  
d  
o  
s  
  
o  
n  
F  
i  
n  
i  
s  
h  
(  
)  
/  
/  
n  
a  
v  
e  
g  
a  
r  
a  
M  
a  
i  
n  
A  
c  
t  
i  
v  
i  
t  
y  
}
```

**P.5 Clases creadas, herencia y sobreescritura de métodos****1 pt**

En el proyecto GearLog se han creado 3 clases Kotlin:

| Clase                  | Descripción                                         |
|------------------------|-----------------------------------------------------|
| <b>SplashActivity</b>  | Activity de presentación al arrancar la app         |
| <b>MainActivity</b>    | Activity principal — pantalla 'Sobre la Aplicación' |
| <b>Theme.kt (obj.)</b> | Función composable que define el tema visual global |

**Análisis de herencia: MainActivity**



La clase MainActivity hereda (extiende) de ComponentActivity, que es la clase base de Jetpack Compose para Activities. La cadena de herencia completa es:

```
MainActivity  
└── ComponentActivity  
└── androidx.activity.ComponentActivity
```

```
n
e
n
t
A
c
t
i
v
i
t
y
L
-
-
F
r
a
g
m
e
n
t
A
c
t
i
v
i
t
y
L
-
-
A
p
p
C
o
m
p
a
t
A
c
t
i
v
i
t
y
L
-
-
A
c
t
i
v
i
```

```
t  
y  
(  
c  
l  
a  
s  
e  
b  
a  
s  
e  
d  
e  
A  
n  
d  
r  
o  
i  
d  
)
```

MainActivity sobrescribe el método onCreate(), que es el punto de entrada del ciclo de vida de cualquier Activity en Android:

```
c  
l  
a  
s  
s  
M  
a  
i  
n  
A  
c  
t  
i  
v  
i  
t  
y  
:  
C  
o  
m  
p  
o  
n  
e  
n  
t  
A  
c  
t  
i  
v  
i  
t  
y
```

```
(  
)  
{  
  
    override  
        fun  
            onC  
                re  
                    at  
                        e  
                            (  
                                s  
                                    a  
                                        v  
                                            e  
                                                d  
                                                    I  
                                                        n  
                                                            s  
                                                                t  
                                                                    a  
                                                                        n  
                                                                            c  
                                                                                e  
                                                                                    S  
                                                                                        t  
                                                                                            a  
                                                                                                t  
                                                                                                    e  
                                                                                                        :  
                                                                                                            B  
                                                                                                                u  
                                                                                                                    n  
                                                                                                                        d  
                                                                                                                            l  
                                                                                                                                e  
                                                                                                                                    ?  
                                                                                                                    )  
                                                                                                    {  
                                                                                                        s  
                                                                                                            u  
                                                                                                                p  
                                                                                                                    e  
                                                                                                                        r  
                                                                                                                            .  
                                                                                                                                o  
                                                                                                                                    n  
                                                                                                                                        C
```

```
re  
a  
t  
e  
(  
s  
a  
v  
e  
d  
I  
n  
s  
t  
a  
n  
c  
e  
S  
t  
a  
t  
e  
)  
/  
/  
l  
l  
a  
m  
a  
d  
a  
a  
l  
p  
a  
d  
r  
e  
  
e  
n  
a  
b  
l  
e  
E  
d  
g  
e  
T  
o  
E  
d  
g  
e  
(  
)  
  
s  
e
```

```
t  
C  
o  
n  
t  
e  
n  
t  
{  
  G  
e  
a  
r  
L  
o  
g  
T  
h  
e  
m  
e  
{  
  S  
o  
b  
r  
e  
L  
a  
A  
p  
l  
i  
c  
a  
c  
i  
o  
n  
S  
c  
r  
e  
e  
n  
(  
)  
}  
}  
}  
}
```

```
E  
l  
m  
o
```

d  
i  
f  
i  
c  
a  
d  
o  
r  
o  
v  
e  
r  
r  
i  
d  
e  
i  
n  
d  
i  
c  
a  
q  
u  
e  
s  
e  
e  
s  
t  
á  
s  
o  
b  
r  
e  
e  
s  
c  
r  
i  
b  
i  
e  
n  
d  
o  
u  
n  
m  
é  
t  
o  
d  
o  
h  
e  
r

edad  
Lallamadas  
super  
Con  
Create  
( )  
esobli  
gator  
riap  
ara  
qu  
e  
Android  
id  
i



n  
i  
c  
i  
a  
l  
i  
c  
e  
c  
o  
r  
r  
e  
c  
t  
a  
m  
e  
n  
t  
e  
e  
l  
e  
s  
t  
a  
d  
o  
i  
n  
t  
e  
r  
n  
o  
d  
e  
l  
a  
A  
c  
t  
i  
v  
i  
t  
y  
a  
n  
t  
e  
s  
d  
e  
e  
j  
e

**P.6** ¿Qué es un Composable en Jetpack Compose?**1 pt**

Un Composable es una función Kotlin anotada con `@Composable` que describe una parte de la interfaz de usuario. En lugar del sistema tradicional de XML + View, Jetpack Compose usa funciones composables que se llaman entre sí para construir la UI de forma declarativa: el desarrollador describe cómo debe verse la pantalla en función del estado, y Compose se encarga de actualizar solo las partes que cambian.

**Composables identificados en GearLog**

| Composable    | Función en GearLog                                                           |
|---------------|------------------------------------------------------------------------------|
| <b>Box</b>    | Contenedor apilado. Se usa para el fondo degradado y para centrar el logo.   |
| <b>Column</b> | Apila elementos en vertical. Organiza todos los elementos de la pantalla.    |
| <b>Text</b>   | Muestra el nombre de la app, la descripción, etiquetas y el copyright.       |
| <b>Card</b>   | Tarjeta Material 3 con fondo oscuro que agrupa la información y descripción. |
| <b>Button</b> | Botón de envío de correo electrónico con icono y texto.                      |
| <b>Icon</b>   | Icono de Email (Icons.Default.Email) dentro del botón de contacto.           |
| <b>Spacer</b> | Espacio vacío para separar elementos verticalmente sin contenido.            |
| <b>Row</b>    | Alinea elementos horizontalmente. Usado en InfoRow para label + valor.       |

### Divider

Línea horizontal separadora entre las filas de información de la Card.

Ejemplo de uso del composable Text en GearLog:

```
@Composable
fun Screen() {
    Column {
        Text("te
```

```
x  
t  
=  
s  
t  
r  
i  
n  
g  
R  
e  
s  
o  
u  
r  
c  
e  
(  
i  
d  
=  
R  
.  
s  
t  
r  
i  
n  
g  
.  
a  
p  
p  
_  
n  
a  
m  
e  
)  
,  
/  
/  
'  
G  
e  
a  
r  
L  
o  
g  
,  
  
f  
o  
n  
t  
S  
i  
z  
e  
=  
3  
6
```

```
.sp, fontweight=FontWeight.ExtraBold, color=Color.White)
Text(
```

```
text = stringResource(id = R.string.app_title_agline), fontSize = 14.sp,
```

```
color = Color.  
color(0xFFEE945600)  
/  
/  
rojo  
corpo  
rativo  
}  
}
```

### **P.7** Función específica del método onCreate()

**1 pt**

El método onCreate() es el primer método del ciclo de vida de una Activity que se ejecuta cuando Android crea la Activity por primera vez (o la recrea tras un cambio de configuración como rotar la pantalla). Es el equivalente al constructor de la Activity, pero gestionado por el sistema Android.

#### **¿Qué hace exactamente en GearLog?**

En MainActivity, onCreate() realiza las siguientes acciones en orden:

- 1. Llama a `super.onCreate(savedInstanceState)` para que Android inicialice el estado interno de la Activity y restaure el estado guardado previamente si existe.
- 2. Llama a `enableEdgeToEdge()` para que el contenido de la app se extienda por debajo de la barra de estado y la barra de navegación del sistema, logrando un diseño inmersivo.
- 3. Llama a `setContent { }` que es la función de Compose que infla la UI. Dentro se establece el tema `GearLogTheme` y se llama al composible raíz `SobreLaAplicacionScreen()`, que construye toda la interfaz visible.

o  
n  
C  
r  
e  
a  
t  
e  
(  
)  
s  
o  
l  
o  
s  
e  
e  
j  
e  
c  
u  
t  
a  
u  
n  
a  
v  
e  
z  
p  
o  
r  
i  
n  
s  
t  
a  
n  
c  
i  
a  
d  
e  
l  
a  
A  
c  
t  
i  
v



i  
t  
y  
(  
s  
a  
l  
v  
o  
r  
e  
c  
r  
e  
a  
c  
i  
ó  
n  
)  
.  
N  
o  
d  
e  
b  
e  
u  
s  
a  
r  
s  
e  
p  
a  
r  
a  
o  
p  
e  
r  
a  
c  
i  
o  
n  
e  
s  
d  
e  
r  
e  
d  
o  
a  
c  
c  
e  
s  
s

o  
a  
b  
a  
s  
e  
s  
d  
e  
d  
a  
t  
o  
s  
q  
u  
e  
p  
u  
e  
d  
a  
n  
b  
l  
o  
q  
u  
e  
a  
r  
e  
l  
h  
i  
l  
o  
p  
r  
i  
n  
c  
i  
p  
a  
l  
.

El ciclo de vida completo de una Activity sigue el orden: onCreate → onStart → onResume → (en uso) → onPause → onStop → onDestroy.

|            |                                                      |
|------------|------------------------------------------------------|
| <b>P.8</b> | <b>Uso del objeto Intent — implícito y explícito</b> |
|------------|------------------------------------------------------|

|             |
|-------------|
| <b>1 pt</b> |
|-------------|

Un Intent es un objeto de Android que representa una intención de realizar una acción. Sirve para comunicar componentes: iniciar Activities, servicios, o pedir al sistema que realice alguna tarea.

Existen dos tipos:

- Intent explícito: especifica exactamente qué componente (clase) debe recibir la acción. Se usa para navegar entre Activities de la misma app.
- Intent implícito: describe la acción a realizar sin especificar quién la ejecuta. El sistema Android decide qué app puede manejarla (puede mostrar un selector si hay varias).

### Intent implícito en GearLog — Envío de correo

En el botón de contacto de MainActivity se usa un Intent implícito con ACTION\_SENDTO para abrir el cliente de correo del dispositivo:

```
val intent = Intent(Intent.ACTION_SENDTO).apply {
```

```
a  
t  
a  
=  
U  
r  
i  
.  
p  
a  
r  
s  
e  
(  
"  
m  
a  
i  
l  
t  
o  
:  
"  
)  
/  
/  
s  
o  
l  
o  
a  
p  
p  
s  
d  
e  
e  
m  
a  
i  
l  
  
p  
u  
t  
E  
x  
t  
r  
a  
(  
I  
n  
t  
e  
n  
t  
.  
E  
X  
T  
R  
A
```

```
-EMAIL, arrayOf("contacto@gearlog.apk")  
putExtra(Intent.EXTRA_S
```

```
U  
B  
J  
E  
C  
T  
,"  
I  
n  
f  
o  
r  
m  
a  
c  
i  
ó  
n  
s  
o  
b  
r  
e  
G  
e  
a  
r  
L  
o  
g  
")  
p  
u  
t  
E  
x  
t  
r  
a  
(  
I  
n  
t  
e  
n  
t  
.  
E  
X  
T  
R  
A  
_  
T  
E  
X  
T  
/  
"  
H
```

```
ola,\n\nMe\npongo\ncon\ncontacto...\n")\ncontext.startActivity(\n    new Intent()\n)
```

```
attach  
chooser(  
intent,  
"Enviar correo"  
)  
)
```

### Ejemplo de Intent explícito en GearLog — SplashScreen a MainActivity

La SplashScreen usa un Intent explícito para navegar a MainActivity cuando acaba el temporizador:

```
//  
// En SplashScreen  
// Actividad  
// y.kt
```



```
s  
t  
a  
r  
t  
A  
c  
t  
i  
v  
i  
t  
y  
(  
I  
n  
t  
e  
n  
t  
(  
t  
h  
i  
s  
@  
S  
p  
l  
a  
s  
h  
A  
c  
t  
i  
v  
i  
t  
y  
,  
M  
a  
i  
n  
A  
c  
t  
i  
v  
i  
t  
y  
:  
:  
c  
l  
a  
s  
s  
.  
j  
a
```

```
v  
a  
)  
)  
f  
i  
n  
i  
s  
h  
(  
)  
/  
/  
c  
i  
e  
r  
r  
a  
S  
p  
l  
a  
s  
h  
A  
c  
t  
i  
v  
i  
t  
y  
p  
a  
r  
a  
q  
u  
e  
n  
o  
q  
u  
e  
d  
e  
e  
n  
l  
a  
p  
i  
l  
a
```

En el futuro, cuando la app tenga una pantalla de detalle de reparación, se podría usar un Intent explícito pasando el ID de la reparación como extra para abrirla directamente.

**P.9** Fichero strings.xml — modificación y efecto**1 pt**

El fichero strings.xml se encuentra en app/src/main/res/values/strings.xml y centraliza todas las cadenas de texto que se muestran en la interfaz de la aplicación. Esto permite:

- Traducir la app a otros idiomas creando carpetas values-es/, values-en/, values-fr/, etc.
- Modificar un texto en un único lugar y que se actualice en toda la app automáticamente.
- Evitar strings duplicadas o hardcodeadas en el código Kotlin.

Contenido del fichero strings.xml de GearLog:

```
<resources>
<string name="app_name">GearLog</string>
```

```
g  
>  
  
<  
s  
t  
r  
i  
n  
g  
n  
a  
m  
e  
=  
"  
a  
p  
p  
t  
a  
g  
l  
i  
n  
e  
"  
>  
T  
U  
T  
A  
L  
L  
E  
R  
E  
N  
E  
L  
B  
O  
L  
S  
I  
L  
L  
O  
<  
/  
s  
t  
r  
i  
n  
g  
>  
  
<  
s  
t  
r
```

```
i  
n  
g  
n  
a  
m  
e  
=  
"  
a  
p  
p  
l  
i  
c  
a  
t  
i  
v  
a  
"  
>  
R  
e  
d  
s  
o  
c  
i  
a  
l  
d  
e  
m  
e  
c  
i  
a  
n  
i  
c  
a  
y  
a  
u  
t  
o  
m  
o  
c  
i  
ó  
n  
<  
/  
s  
t  
r  
i  
n  
g  
<
```

```
<
s
t
r
i
n
g
n
a
m
e
=
"
a
p
p
_
v
e
r
s
i
o
n
"
>
1
1
.
0
.
0
<
/
s
t
r
i
n
g
>
<
s
t
r
i
n
g
n
a
m
e
=
"
a
p
p
_
q
u
e
s
a
```

```
r  
r  
o  
l  
l  
a  
d  
o  
r  
"  
>  
V  
a  
l  
e  
n  
t  
i  
n  
U  
r  
d  
i  
l  
l  
o  
<  
/  
s  
t  
r  
i  
n  
g  
>  
  
<  
s  
t  
r  
i  
n  
g  
n  
a  
m  
e  
=  
"  
a  
p  
p  
l  
i  
c  
a  
t  
i  
o  
n
```

n  
">  
G  
e  
a  
r  
L  
o  
g  
e  
s  
u  
n  
a  
r  
e  
d  
s  
o  
c  
i  
a  
l  
p  
e  
n  
s  
a  
d  
a  
p  
a  
r  
a  
  
a  
f  
i  
c  
i  
o  
n  
a  
d  
o  
s  
y  
p  
r  
o  
f  
e  
s  
i  
o  
n  
a  
l  
e  
s  
d  
e



```
l  
a  
m  
e  
c  
á  
n  
i  
c  
a  
.  
.  
.  
<  
/  
s  
t  
r  
i  
n  
g  
>  
  
<  
s  
t  
r  
i  
n  
g  
n  
a  
m  
e  
=  
"  
b  
t  
n  
_  
c  
o  
n  
t  
a  
c  
t  
a  
r  
"  
>  
c  
o  
n  
t  
a  
c  
t  
a  
r  
p  
o  
r
```

```
c  
o  
r  
r  
e  
o  
<  
/  
s  
t  
r  
i  
n  
g  
>  
  
<  
s  
t  
r  
i  
n  
g  
n  
a  
m  
e  
=  
"  
a  
p  
p  
-  
c  
o  
p  
y  
r  
i  
g  
h  
t  
"  
>  
©  
2  
0  
2  
5  
V  
a  
l  
e  
n  
t  
í  
n  
U  
r  
d  
i  
l  
l  
o
```

```
o . G  
e a r  
L o g  
v l  
. 0  
. 0  
< /  
s t r  
i n g  
> <  
/ r  
e s  
o u  
r c  
e s  
>
```

### Modificación de ejemplo

Si cambiamos el valor de app\_tagline de 'TU TALLER EN EL BOLSILLO' a 'COMPARTE TU PASIÓN POR LOS COCHES', este cambio se refleja automáticamente en todas las pantallas donde se use `stringResource(R.string.app_tagline)`, sin tocar ningún fichero Kotlin.

```
< !  
-  
-  
A  
n  
t  
e  
s  
-  
-  
>  
<  
s  
t  
r  
i  
n
```

```
g
n
a
m
e
=
"
a
p
p
-
t
a
g
g
l
i
n
e
"
>
T
U
T
A
L
L
E
R
E
N
E
L
B
O
L
S
I
L
L
O
<
/
s
t
r
i
n
g
>

<
-
-
-
D
e
s
p
u
e
s
-

```

```
-  
>  
<  
s  
t  
r  
i  
n  
g  
n  
a  
m  
e  
=  
"  
a  
p  
p  
l  
i  
c  
a  
g  
l  
i  
n  
e  
"  
>  
C  
O  
M  
P  
A  
R  
T  
E  
T  
U  
P  
A  
S  
I  
Ó  
N  
P  
O  
R  
L  
O  
S  
C  
O  
D  
H  
E  
S  
<  
/  
s  
t  
r  
i  
n
```

```
g  
>
```

**P.1  
0****Directorio mipmap e ic\_launcher.xml****1 pt**

El directorio mipmap/ se usa específicamente para almacenar los iconos del launcher de la aplicación (el icono que aparece en la pantalla de inicio del dispositivo y en el listado de apps). A diferencia del directorio drawable/, los recursos mipmap nunca se escalan hacia abajo por el sistema, lo que garantiza que el icono se vea nítido en todos los dispositivos.

**Subdirectorios mipmap por densidad**

| Directorio         | Densidad – Tamaño del icono             |
|--------------------|-----------------------------------------|
| mipmap-mdpi/       | ~160 dpi — 48×48 px                     |
| mipmap-hdpi/       | ~240 dpi — 72×72 px                     |
| mipmap-xhdpi/      | ~320 dpi — 96×96 px                     |
| mipmap-xxhdpi/     | ~480 dpi — 144×144 px                   |
| mipmap-xxxhdpi/    | ~640 dpi — 192×192 px                   |
| mipmap-anydpi-v26/ | API 26+ — icono adaptativo (vector XML) |

**Análisis de la línea en ic\_launcher.xml**

```
<b  
a  
c  
k  
g  
r  
o  
u  
n  
d  
a  
n  
d  
r  
o  
i  
d  
:  
d  
r  
a  
w  
a  
b  
l
```

```
e  
=  
"@  
d  
r  
a  
w  
a  
b  
l  
e  
/  
i  
c  
_  
l  
a  
u  
n  
c  
h  
e  
r  
_  
b  
a  
c  
k  
g  
r  
o  
u  
n  
d  
"  
/  
>
```

Esta línea forma parte del icono adaptativo (Adaptive Icon), introducido en Android 8.0 (API 26). Define la capa de fondo del icono. El atributo `android:drawable="@drawable/ic_launcher_background"` hace referencia al fichero `ic_launcher_background.xml` del directorio `drawable/`, que en GearLog define un degradado lineal de color azul oscuro (`#1A1A2E` → `#0F3460`). El sistema Android puede recortar esta capa de distintas formas según el launcher del fabricante (círculo, cuadrado redondeado, squircle, etc.), por eso se usan dos capas separadas: fondo (background) y primer plano (foreground).

**P.1**  
**1****Función del AndroidManifest.xml y declaración de Activities****1 pt**

El fichero `AndroidManifest.xml` es el fichero de configuración central de toda aplicación Android. Se encuentra en `app/src/main/AndroidManifest.xml`. Es el primer fichero que lee el sistema operativo Android cuando va a instalar o ejecutar una aplicación, y contiene toda la información que Android necesita para gestionar la app.

## Funciones principales del AndroidManifest.xml

- Declara el package único de la aplicación (com.valentinurdillo.gearlog).
- Lista todas las Activities, Services, BroadcastReceivers y ContentProviders de la app.
- Indica qué Activity es el punto de entrada (LAUNCHER) al arrancar la app.
- Declara los permisos que la aplicación necesita del sistema (internet, cámara, etc.).
- Especifica la versión mínima de Android soportada (minSdk) y la versión objetivo (targetSdk).
- Define el icono, nombre, tema y otras propiedades globales de la aplicación.

## ¿Por qué hay que declarar cada Activity?

Android necesita conocer todas las Activities de la app antes de ejecutarla. Si una Activity no está declarada en el Manifest, el sistema Android no sabe de su existencia y lanzará una excepción `ActivityNotFoundException` en tiempo de ejecución cuando se intente abrir, cerrando la aplicación con un crash.

```
<!-- Si no se declara la actividad, se lanzará una excepción ActivityNotFoundException -->
```



```
c  
a  
u  
s  
a  
c  
r  
a  
s  
h  
-  
-  
>  
<  
a  
c  
t  
i  
v  
i  
t  
y  
  
a  
n  
d  
r  
o  
i  
d  
:  
n  
a  
m  
e  
=  
"  
.M  
a  
i  
n  
A  
c  
t  
i  
v  
i  
t  
y  
"  
  
a  
n  
d  
r  
o  
i  
d  
:  
e  
x  
p
```

```
o  
r  
t  
e  
d  
=  
"  
f  
a  
l  
s  
e  
"  
  
a  
n  
d  
r  
o  
i  
d  
:  
s  
c  
r  
e  
e  
n  
O  
r  
i  
e  
n  
t  
a  
t  
i  
o  
n  
=  
"  
p  
o  
r  
t  
r  
a  
i  
t  
"  
/  
>
```

**P.1**  
**2**

**Cláusulas del fichero de manifiesto**

**1 pt**

### **Permisos solicitados por GearLog**

En el AndroidManifest.xml de GearLog se declara el siguiente permiso:

```
<uses-permission:android:name="android.permission.INTERNET"/>
```

Este permiso es necesario para que la app pueda realizar peticiones de red (en el futuro, para consultar la API pública de la NHTSA y cargar imágenes de usuarios). Es un permiso normal (no

peligroso), por lo que no hace falta solicitarlo en tiempo de ejecución; basta con declararlo en el Manifest.

### Explicación de las cláusulas

| Cláusula                                        | Significado                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>android:allowBackup="true"</code>         | Permite que Android realice copias de seguridad automáticas de los datos de la app (SharedPreferences, bases de datos, ficheros) en la cuenta de Google del usuario. Si se desinstala y reinstala la app, los datos se restauran automáticamente. Puede desactivarse por razones de privacidad ( <code>allowBackup="false"</code> ).                |
| <code>android:icon="@mipmap/ic_launcher"</code> | Especifica el icono que representa la app en el launcher, en el listado de apps instaladas y en el gestor de aplicaciones del sistema. La referencia <code>@mipmap/ic_launcher</code> apunta al directorio <code>mipmap/</code> , y Android seleccionará automáticamente la versión con la densidad adecuada para el dispositivo en que se ejecute. |

## 4. Webgrafía

---

Fuentes consultadas para la elaboración de esta práctica:

**[1] Android Developers – Activity lifecycle:**

<https://developer.android.com/guide/components/activities/activity-lifecycle>

**[2] Android Developers – Jetpack Compose basics:**

<https://developer.android.com/develop/ui/compose/documentation>

**[3] Android Developers – Intents and intent filters:**

<https://developer.android.com/guide/components/intents-filters>

**[4] Android Developers – App manifest overview:**

<https://developer.android.com/guide/topics/manifest/manifest-intro>

**[5] Android Developers – Adaptive icons:**

[https://developer.android.com/develop/ui/views/launch/icon\\_design\\_adaptive](https://developer.android.com/develop/ui/views/launch/icon_design_adaptive)

**[6] Android Developers – String resources:**

<https://developer.android.com/guide/topics/resources/string-resource>

**[7] Android Developers – SplashScreen API:**

<https://developer.android.com/develop/ui/views/launch/splash-screen>

**[8] NHTSA Vehicle API – Documentación oficial:** <https://api.nhtsa.gov>

**[9] Kotlin documentation – Classes and inheritance:** <https://kotlinlang.org/docs/classes.html>

**[10] GitHub – Awesome Public APIs:** <https://github.com/public-apis/public-apis>